

ROCKSTORE-SISHIBRIDO

Documento de Integración y Seguridad: Detalles de la implementación del puente de autenticación JWT entre el servicio de identidad (C#) y el servicio lógico (Python).

1. Microservicio de Identidad (C# .NET)

Este servicio es el guardián del sistema. Se encarga de crear usuarios, encriptar contraseñas y emitir los tokens JWT.

1.1 Configuración de la Base de Datos (AppDbContext)

Se implementó Entity Framework Core con SQLite. Destaca la inclusión de un **Filtro de Consulta Global (Query Filter)** para manejar el borrado lógico (Soft Delete) automáticamente.

```
modelBuilder.Entity<User>().HasQueryFilter(u => u.DeletedAt == null);
```

1.2 Generación de JWT (AuthController)

El controlador de autenticación cuenta con dos endpoints principales:

- **POST /api/auth/register:** Hashea la contraseña usando BCrypt y guarda al usuario.
- **POST /api/auth/login:** Valida credenciales y genera un Token JWT firmado con una clave simétrica (HMAC-SHA256). El token incluye Claims como el Email, Role y el Id del usuario.

2. Microservicio Lógico (Python FastAPI)

Este servicio gestiona el e-commerce. Sus rutas privadas confían en la firma del token generado por C# para autorizar peticiones.

2.1 Configuración ORM (SQLAlchemy)

Se configuraron los modelos de base de datos relacional (Products, Customers, Orders, OrderItems) en SQLite. La tabla Customers utiliza user_id como llave foránea lógica hacia la base de datos de C#.

2.2 Puente de Seguridad JWT (auth.py)

Se creó una dependencia de seguridad utilizando PyJWT y HTTPBearer. Esta función intercepta las peticiones, lee la cabecera Authorization y verifica criptográficamente la firma del token.

```
payload = jwt.decode(
    token,
    SECRET_KEY,
    algorithms=[ALGORITHM],
    audience=AUDIENCE,
    issuer=ISSUER
)
# Extracción del ID mapeado por C#
user_id_claim = "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/nameidentifier"
user_id = payload.get(user_id_claim)
```

2.3 Rutas Protegidas (main.py)

FastAPI implementa el candado inyectando la dependencia en las rutas. Si el token falta, es inválido o expiró, rechaza la petición con un error 401 Unauthorized.

```
@app.get("/api/perfil")
def obtener_perfil_seguro(user_id: str = Depends(get_current_user)):
    return {"user_id_desde_csharp": user_id}
```